

When Is It Safe to Stop Looking at the Image?

A measured visual token access threshold in two vision language models, and what it means for pruning

P. S. Kedar

Supervised by Prof. Divya Saxena | Mentored by Aditya Sharma, PhD Scholar
School of AI and Data Engineering, IIT Jodhpur | Phase 3 Report, September 2026
Subject paper: Balanced Token Pruning (NeurIPS 2025, arXiv:2505.22038)

1. Summary

Vision language models are expensive to run because every image becomes thousands of tokens. Pruning methods throw most of those tokens away as the model runs. Balanced Token Pruning (BTP) is one such method, published at NeurIPS 2025, and its released code also deletes every remaining image token at a fixed layer to save more compute.

We set out to explain why that code destroys the model's ability to read text inside an image. The answer turned out to rest on a quantity that we did not find measured in the literature we reviewed for this report.

Three findings, at three levels

1. A model-level finding. There is a threshold. In both models we tested, performance stays sensitive to visual tokens until roughly three quarters of the way through the decoder, and then abruptly stops being so. In the TextVQA setting we tested, deleting all remaining image tokens before that depth caused a sharp collapse, while deletion after it had a much smaller effect. The threshold is sharp and differs by architecture: InternVL2-2B needs the image through 75 per cent of its decoder, Qwen2.5-VL through 80 per cent.
2. An implementation-level finding. The released code sits one layer on the wrong side of it. BTP allows Qwen 22 layers of access against a threshold of 22.5. Twenty-two layers retains 27 per cent of baseline reading accuracy; twenty-three retains 75 per cent. One layer is worth 47.7 points.
3. An evaluation-level finding. With the fault present, the code still scores 96.1 per cent of baseline across five of the six benchmarks the paper reports, inside the 96 to 98 per cent it claims. Not one of those six asks the model to read text from an image.

The two effects are separable and we keep them apart throughout. On TextVQA and DocVQA the deletion accounts for most of the loss. On ChartQA it does not: there the pruning itself costs twice what the deletion does, so the fault is not the main story on every benchmark.

2. The Problem

Earlier phases reproduced the published results on three models within about a point. We then tested on text-heavy benchmarks, which the paper does not report.

- LLaVA-1.5-7B behaved sensibly. TextVQA fell from 46.1 to 40.0, a loss of 6.1 points.
- Qwen2.5-VL-7B collapsed. TextVQA fell from 86.2 to 23.3, a loss of 62.9 points. DocVQA fell from 94.7 to 19.2, ChartQA from 76.8 to 29.0.
- AI2D, a diagram benchmark that rarely needs text read word by word, barely moved at 86.4 to 79.6. It served as our control.

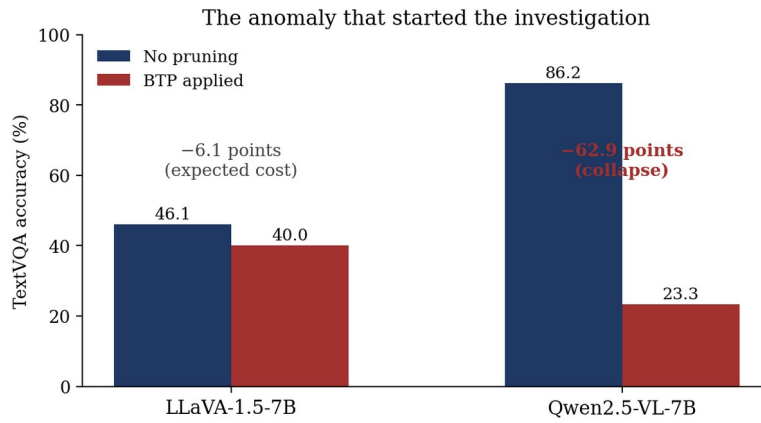


Figure 1. The same method on two model families, two very different outcomes.

3. What We Ruled Out

We listed the candidate causes first and built a test capable of killing each. Four returned negative before the real cause appeared.

Hypothesis tested	Observation	Interpretation
Selection rule, ablated at a fixed budget: BTP, diversity, attention, random	All four 17.3 to 19.4, inside the error bars	Not the selector
Are text patches near-duplicates in feature space?	Text-text 0.379, background-background 0.438	Our premise was wrong
Does the amount pruned matter? Swept retention from 100 to 12.5 per cent	Retaining 90 per cent already gave the full drop, 86.2 to 23.3. Retaining 12.5 per cent gave 23.3, the same	Not the amount
Force every text token to survive, using ground-truth OCR	0.197 oracle against 0.197 control	Not even a perfect selector

Table 1. The four negative results, in the order they were run.

All four were measuring the same floor. Each changed something upstream of the deletion and then scored an answer produced downstream of it, where no image token survives.

4. The Cause and the Causal Intervention

The clue came from a control we added almost as an afterthought: a setting at 100 per cent retention, where the pruning stages keep every token and should behave exactly like no pruning at all. It scored 0.178, the same as every other budget. We verified the retention value was genuinely reaching the worker process. Something downstream of the budget was destroying the image.

Searching the model file for every branch that touches the hidden-state sequence turned up one candidate.

```

elif layer_index == self.end_layer:      # self.end_layer = 23
    remain_sys_index = torch.tensor(list(range(self.img_start_idx)))
    remain_text_index = torch.tensor(list(range(img_end, total_length)))

    # there is no remain_img_index in the line below
    all_remained_index = torch.cat((remain_sys_index, remain_text_index))
    hidden_states = hidden_states[:, all_remained_index, :]

```

Where this code lives	
Repository	NeurIPS2025-Balanced-Token-Pruning, the authors' release
Commit	9682db0
File	qwen-2.5-vl/modeling_qwen2_5_vl.py
Class and method	Qwen2_5_VLModel.forward, class at line 1115, method at line 1184
Constants	lines 1140 to 1144, where end_layer is set to 23
The branch	line 1385
State when read	unmodified. We copy this file over the installed transformers module and restore the original afterwards, so the released file is never edited in place.

Table 2. Provenance of the code examined. Line numbers refer to the copy in our repository under repo/qwen-2.5-vl/.

The concatenation joins system tokens and text tokens. There is no `remain_img_index`. Every image token that survived three stages of careful pruning is deleted at layer 23, always. Just above it sits a commented-out version that would have kept some by attention score.

An explanation that fits the data is not yet evidence of cause. To test it we changed one integer, so the branch can never fire, and left everything else alone including the full 12.5 per cent pruning.

Arm	TextVQA	AI2D (control)
No pruning (baseline)	0.8565	0.8750
BTP as released	0.1725	0.8150
BTP with the layer-23 deletion switched off	0.7860	0.8100

Table 3. Job 416735, 200 samples. One line changed, accuracy moves from 0.1725 to 0.7860. The baseline of 0.8565 differs slightly from the 86.2 quoted elsewhere because this intervention used a separate 200 sample subset.

- An intervention, not an observation. One variable changed, everything else fixed.
- The flat curve beforehand was the giveaway: 100, 50 and 12.5 per cent retention all scored 0.178, because nothing survived layer 23 in any of them.
- AI2D hardly moves across all three arms, so this is not a change that simply lifts every score.

5. What the Paper Specifies

Deleting visual tokens at depth is a legitimate design choice, so calling it a fault without checking would be careless. Appendix 7.3 of the paper covers the final pruning stage for each model family.

For LLaVA-v1.5-7B, LLaVA-v1.5-13B, and LLaVA-v1.6-7B, we divide the pruning process into five stages ... In the final stage, all tokens are discarded to maximize inference speed. For Qwen2.5-VL, since its image token processing can be clearly divided into two stages, we retain 25% of the tokens in the fourth stage and 12.5% in the final stage to preserve model performance.

- Discarding everything at the end is a real part of BTP, and the stated reason is speed.
- It is specified for the LLaVA family only.
- For Qwen2.5-VL the paper asks for the opposite: keep 12.5 per cent, and the stated reason is, in the authors' own words, to preserve model performance.

- In the released implementation we examined, the LLaVA-style deletion is applied to Qwen as well, with no model-specific check.

Two obvious objections do not hold. This is not the VTW baseline leaking in, since the code keeps VTW separately with its own flag and layer number. And it is not implied by the method section, which defines pruning at exactly three layers, each keeping a subset.

6. The Threshold

Knowing that layer 23 was fatal raised a better question. Is depth itself the problem, or is 23 simply the wrong depth? So we swept it. All visual tokens are deleted at a given layer, graded pruning is switched off, and nothing else varies. We did this on Qwen2.5-VL-7B and again on InternVL2-2B, a model chosen because it differs on every axis that could confound: a 24 layer InternLM2 backbone, ordinary one-dimensional RoPE rather than M-RoPE, and 448 pixel dynamic tiling rather than a merged patch grid.

How layers are counted here

Everything below is expressed as the number of layers that still had access to the image, counting from the first. Deleting before layer 12 of 28 means twelve layers had access and sixteen did not.

This is not the same as the index a method writes in its code. BTP counts decoder layers from one and deletes before the named layer, so its end_layer of 23 leaves twenty-two layers with access. Our sweeps and tool count from zero. The two differ by one, and an earlier draft of this report conflated them.

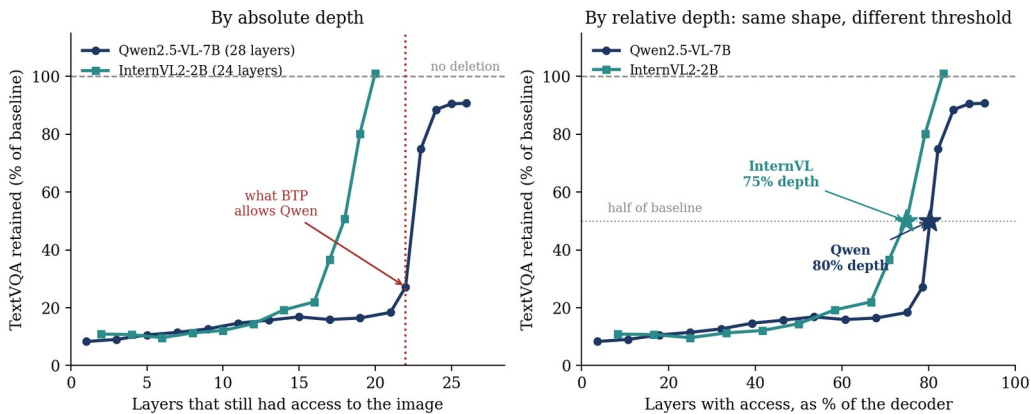


Figure 2. Left, by absolute layer. Right, by relative depth. Both models hold a flat floor near 10 to 18 per cent, then recover sharply.

Both curves have the same shape. Accuracy sits on a floor while the deletion happens early, then rises steeply once it happens late enough. The results indicate that performance remains sensitive to visual token access until a particular depth, after which the effect of complete deletion becomes much smaller. We measure the effect of removing the tokens, not what the model does with them, so we describe the sensitivity rather than infer an internal mechanism.

Model	Decoder layers	Layers needing the image	Relative depth
InternVL2-2B	24	18.0	74.8 %
Qwen2.5-VL-7B	28	22.5	80.3 %

Table 4. The number of layers that still need visual tokens. Counted as layers with access, not as the index of the deletion, which differ by one.

The fractional values need defining. We take the threshold to be the point at which accuracy crosses half of the unpruned baseline, and we obtain it by linear interpolation between the two measured layers that bracket that crossing. For Qwen those are 22 layers of access at 27.2 per cent and 23 at 74.8 per cent, giving 22.48. The bisection described later narrows the bracket; it does not change the definition. A different criterion, say 90 per cent of baseline, would move both numbers, so the two models should be compared against each other rather than read as absolutes.

6.1 What this costs BTP

BTP's released configuration sets `end_layer` to 23, which leaves 22 of Qwen's 28 layers with access to the image. Our sweep places the threshold at 22.5. The shipped constant therefore lands on the last value at which total discard is still ruinous.

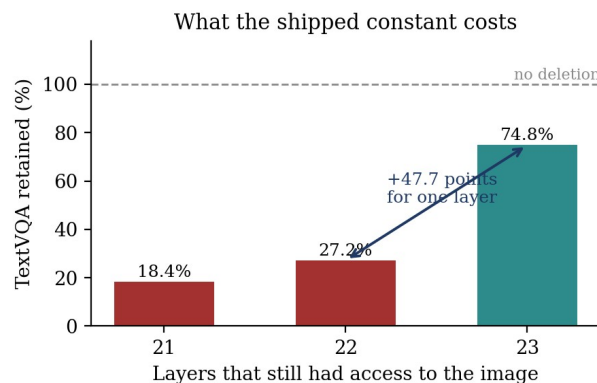


Figure 3. One layer either side of the threshold, measured on TextVQA at 500 samples. BTP allows 22.

Had the constant allowed one more layer, the same code would have scored about 0.645 instead of 0.234 on TextVQA. These two figures come from the 500 sample threshold sweep and are therefore not the same numbers as the 200 sample intervention in Table 2, which gave 0.1725 and 0.7860 under a different configuration. This is not a subtle tuning question. It is a cliff, and the released value sits at the bottom of it. The threshold here is measured on TextVQA only; whether it sits at the same depth for documents or charts is the obvious next experiment and is not yet done.

6.2 The measurement does not depend on the instrument

Both sweeps above edit the model source, separately for each architecture, which is not something anyone else can reuse. So we also built the measurement as a tool that deletes the tokens with a forward hook and touches no source at all, and ran it on Qwen to see whether it agrees.

Layers with access	By source patch	By forward hook
13	15.7 %	17.0 %
22	27.2 %	27.5 %
23	74.8 %	75.8 %
24	88.5 %	86.3 %

Table 5. The same threshold from two independent instruments on different samples. Job 423959.

The two agree within a couple of points across the curve, on different sample sets. On this model, then, the threshold is not an artefact of how it was measured. We have run the comparison on one model, so this establishes that the tool works, not that it will agree everywhere. The tool needs no training, no calibration set and no changes to the model, and a coarse sweep with bisection locates the threshold in about seven evaluations.

This comparison also corrected an error of ours. BTP counts decoder layers from one, our hook counts from zero, and both delete before the named layer runs, so the same experiment carries two different labels. An earlier draft quoted Qwen's threshold as 84 per cent by taking the raw layer index rather than the number of layers with access. Measured consistently it is 80 per cent. The figures and tables above use layers with access throughout.

6.3 One difference we did not expect

The two models do not behave identically at the top of the curve. InternVL recovers fully, reaching 101 per cent of baseline with 20 of its 24 layers given access, four still to run. Qwen does not. Its accuracy plateaus at about 90.7 per cent even with 26 of its 28 layers given access, which suggests that visual information may remain useful very late in the decoder for some subset of examples. We cannot say from this experiment which examples those are, or how many. We report it because it was not predicted and it bears on how safe any late-stage discard can be.

7. Results Across the Suite

With the mechanism understood, we evaluated a corrected arm across the benchmarks used in our comparison. Baseline and as-released numbers were reused from earlier phases with matching sample limits, so only the corrected arm needed fresh GPU time.

Benchmark	Baseline	As released	Corrected	Kept
TextVQA	86.2	23.3	80.5	93.4 %
DocVQA (ANLS)	94.7	19.2	76.8	81.1 %
ChartQA	76.8	29.0	45.4	59.1 %
AI2D (control)	86.4	79.6	79.2	91.7 %
GQA	60.9	55.9	58.8	96.5 %
POPE (accuracy)	87.6	86.2	86.3	98.5 %
MME (perception)	1674.5	1658.7	1651.2	98.6 %
MMBench-EN	83.7	79.3	79.0	94.5 %
ScienceQA-Image	88.1	85.1	85.0	96.5 %

Table 6. Qwen2.5-VL-7B. Kept is the corrected score as a percentage of the corresponding baseline. Highlighted rows need the model to read.

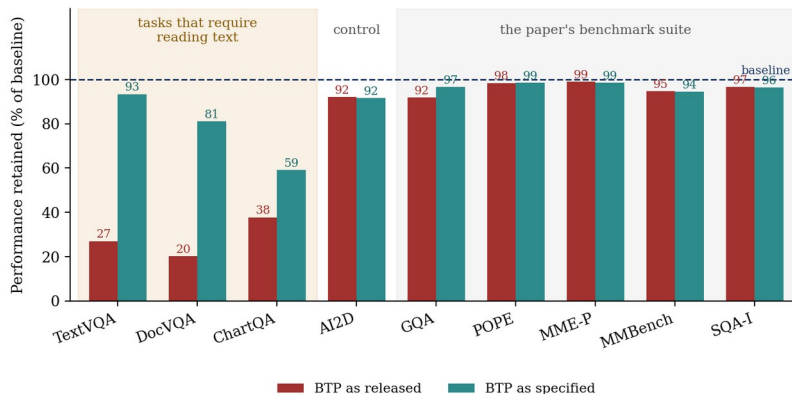


Figure 4. The two arms are indistinguishable everywhere except on the three reading tasks.

7.1 The benchmarks cannot see the fault

The paper reports six benchmarks: GQA, MME, MMBench, POPE, ScienceQA and MM-Vet. We evaluated five of them. MM-Vet is omitted because it is scored by a language model judge, which would add a second source of variance to a comparison meant to isolate one line of code.

Across all five, the released arm and the corrected arm are close: POPE 86.2 against 86.3, MME 1659 against 1651, MMBench 79.3 against 79.0, ScienceQA 85.1 against 85.0, and GQA 55.9 against 58.8. GQA shows the largest gap of the five at 2.9 points, still an order of magnitude smaller than the reading tasks.

Averaged across those five, the released code retains 96.1 per cent of baseline against a claimed 96 to 98 per cent. The corrected arm gives 96.9 per cent. The fault does not merely slip past individual benchmarks; it slips past the average. An evaluation restricted to that suite could therefore classify the implementation as successful, despite the failure on text-reading tasks. On the three reading tasks the same two arms score 28.4 and 77.9 per cent.

7.2 Separating the two effects

The three arms allow the loss to be split. The corrected arm still prunes to 12.5 per cent; it simply does not delete the remainder. So the gap between baseline and corrected is the cost of pruning, and the gap between corrected and released is the cost of the deletion. They are different quantities and should not be quoted as one.

Benchmark	Cost of pruning	Cost of the deletion	Which dominates
TextVQA	5.7	57.2	deletion, 10 to 1
DocVQA	17.9	57.6	deletion, 3 to 1
ChartQA	31.4	16.4	pruning, 2 to 1
AI2D (control)	7.2	0.4	pruning
GQA	2.1	2.9	neither, both small
POPE	1.3	0.1	neither, both small
MMBench-EN	4.7	0.3	neither, both small
ScienceQA-Image	3.1	0.1	neither, both small
MME (perception)	23.3	7.5	neither, see note

Table 7. Points lost to each effect on Qwen2.5-VL-7B. The deletion dominates on TextVQA and DocVQA; on ChartQA the pruning itself is the larger problem. MME is scored out of roughly 2000 rather than 100, so its two figures are not comparable with the rest of the column; as a fraction of its own baseline they are 1.4 and 0.4 per cent. Where the deletion figure would be negative, meaning the released arm scored marginally higher, we give its magnitude.

ChartQA is the row worth pausing on. There the deletion costs 16.4 points and the pruning costs 31.4, so the fault is not the main story on that benchmark. Any claim that the layer deletion explains the collapse holds for scene text and documents, and does not hold for charts. We had originally written the deletion as the dominant cause throughout, which this table shows to be too broad.

8. Pruning Has a Cost of Its Own

A one-line explanation is tempting and easy to overstate. Graded pruning to 12.5 per cent, with no deletion at all, still costs something, and the cost tracks how much reading the task needs. We measured this on InternVL2-2B, where we implemented the pruning ourselves and where no fault is present.

Benchmark	Baseline	Pruned to 12.5 %	Kept
TextVQA	0.716	0.468	65.4 %
DocVQA (ANLS)	0.835	0.303	36.3 %
ChartQA	0.586	0.398	67.9 %
AI2D (control)	0.746	0.728	97.6 %
GQA	0.592	0.546	92.2 %
POPE (accuracy)	0.888	0.848	95.5 %
MMBench-EN	83.4	81.0	97.1 %

Table 8. InternVL2-2B, graded pruning only, no deletion. Kept is the pruned score as a percentage of the corresponding baseline. Reading tasks lose roughly eight times as much as the standard suite reports.

The text suite averages 56.5 per cent of baseline while the standard suite averages 94.9 per cent. There is no bug here at all. The pruning is correct and behaves exactly as designed, and the benchmark suite still reports about 95 per cent while the model has lost close to half its reading ability.

The point

On Qwen2.5-VL there is a fault, and the suite cannot see it. On InternVL2-2B there is no fault, and the suite still cannot see what pruning costs. The blind spot is not a property of one bad implementation. It is a property of the evaluation protocol.

One caveat belongs with this table. Our InternVL selector implements the diversity component only; the attention component is omitted because InternLM2's FlashAttention-2 path does not expose attention scores. In a probe of our own, on a single image, attention-based picks landed on text 36.0 per cent of the time against a 28.8 per cent text share, which suggests attention favours text somewhat. On that basis these figures are likely an upper bound on what BTP itself would cost rather than a like-for-like comparison, though one image is not a firm result. The direction and the asymmetry hold; the magnitude may be inflated.

9. Limitations

- We cannot say what the authors intended, or which settings produced their published table. Our claim is about reproducibility, not about anyone being wrong.
- Two models is two models. The threshold is sharp in both and appears at a similar relative depth, but three points would make a trend and we have two.
- The threshold is measured on TextVQA alone. Whether it sits at the same depth for document or chart understanding is untested.
- The InternVL implementation is ours, not the authors'. It uses the diversity selector only, with layer numbers scaled across from Qwen rather than calibrated.
- The oracle experiment was run at one budget only, after an unexplained slowdown forced us to abandon the sweep.
- Runs use a 500 sample subset. Limits match across arms so comparisons hold, but they are not directly comparable to full-dataset figures published elsewhere.
- **Two claims withdrawn.** Phase 2 blamed the collapse on text having almost no redundancy; our own measurement refutes the premise, since text patches are less alike than background

patches, 0.379 against 0.438. And an early result suggesting that surviving text coverage predicts a correct answer vanished at a larger sample. Neither should be quoted.

10. Conclusion

Visual token pruning methods decide, somewhere in their code, a layer at which the model no longer needs to look at the image. That decision is usually a constant. Our measurements on two architectures indicate that this threshold can be sharp and architecture-dependent: about three quarters of the way through InternVL2-2B, and about four fifths through Qwen2.5-VL. We did not identify a study, in the literature reviewed for this report, that measures it explicitly. The benchmarks these methods are validated on did not detect it here, because not one of them asks the model to read.

Stated as three separate claims, at three different levels:

- **Implementation.** In the released BTP code we examined, all remaining image tokens are deleted at a point that is damaging for Qwen2.5-VL, and the paper's own appendix specifies otherwise for that model.
- **Model.** The two models we tested show a sharp change in sensitivity to visual token access, at different decoder depths.
- **Evaluation.** Standard benchmark evaluation did not reveal the damage that visual token pruning does to text reading, in either the faulty case or the correct one.

The third is the one that generalises furthest, and it does not depend on the first being true. Two practical recommendations follow.

- **Measure the threshold before choosing a pruning layer.** The sweep is cheap and the tool is written: a coarse pass plus bisection finds the threshold in about seven evaluations, using forward hooks, with no training, no calibration set and no changes to the model source.
- **Report at least one text-reading benchmark.** Not because reading matters more than reasoning, but because it is the ability most sensitive to losing visual evidence, and therefore the best early warning. Methods should be measured where they are most likely to break.

10.1 Next steps

- Settle the framing with the mentor: implementation defect, or reproducibility finding.
- Extend the threshold measurement to DocVQA and ChartQA, and ideally to a third architecture.
- Correct the earlier report and deck, publish one tidy repository release, and begin the manuscript.

10.2 A note on method

Four experiments returned negative before the cause was found, and one hypothesis was refuted by a measurement we had predicted would confirm it. An earlier draft of this report concluded the effect was specific to Qwen, on the strength of a single measurement in InternVL; the sweep showed that to be wrong. We record this because the negatives are what ruled out the alternatives, and because the pattern they formed, four unrelated changes all landing on the same number, was itself the clue. A flat curve is not a disappointing result. It is a signature.

Every number here is traceable to a marked entry in the results log. Principal jobs: 411681, 412401, 414662, 416734, 416735, 416896 to 416904, 417968, 418974 to 419003, 419005 to 419012, 421205 to 421218, 421231 to 421237.